CONFIDENTIAL · TECHNICAL EVALUATION

Nur Food POS

Technical Audit and Acquisition Readiness Report

Assessment date: 2 November 2026

Repository: `food_software` (local checkout reviewed)

Live URL supplied: https://food-software-sigma.vercel.app/

Purpose: Technical evaluation for a potential exclusive source-code acquisition.

Status: Independent source review and local verification; no production data was accessed or changed.

1. Executive Summary

Nur Food POS is a polished browser-based restaurant POS prototype with a broad interface, bilingual labels, catalog and cart workflows, receipt/printing code, inventory and report screens, and an extensive set of local regression checks. The browser implementation is principally a client-side application whose operational records are stored in each browser's `localStorage`.

The repository also contains a Laravel 12 API and database schema. This backend is a separate implementation and is not the runtime used by the supplied Vercel deployment: root `vercel.json` builds a Vite static site, while checkout sends a secondary, best-effort request to root `/api/orders`. That Vercel handler validates role/order type but does not persist the order. Authentication sessions in that handler live in process memory, which is not a durable session store for serverless execution. The Laravel backend has database persistence and passing local feature tests, but its deployment and frontend integration are not configured here.

Maturity classification: Prototype / demonstration MVP, not production-ready. The UI and important local workflows are implemented and regression-tested, but financial records are device-local, production order persistence is absent, authentication reliability depends on ephemeral process state in the deployed endpoint, and several reports contain seeded/demo values. The codebase is not ready for a safe independent commercial handover without backend integration, durable infrastructure, security hardening, operational procedures, and legal ownership review.

During this audit, three verified defects were corrected: the sample order's VAT-inclusive figures were inconsistent, cross-tenant order history was not scoped, and shift endpoints did not enforce tenant ownership. After the changes, the Vite build succeeded, the POS suite passed 574 checks, and Laravel passed 12 tests with 60 assertions. Dependency advisory scans could not reach npm/Packagist, so vulnerability status remains unverified.

2. Product Overview

The root application is a Vite-built single-page POS interface with additional static pages (`dashboard.html`, `new_shift.html`, and `splash.html`). It covers menu browsing, cart edits, checkout/tender allocation, receipt generation, printing, order lists, inventory tools, shift workflows, and reports. The user interface includes English and Arabic strings and RTL styling.

The Laravel API under `backend/` provides PIN login, token session endpoints, order/shift/customer routes, migrations, seed data, and feature tests. No deployment manifest connects this Laravel service to the Vercel project. The root's `api/` handlers are a distinct Node.js serverless-style API and are also emulated by Vite middleware locally.

Evidence includes `index.html`, `js/`, `api/`, `backend/routes/api.php`, `backend/app/Http/Controllers/Api/`, `backend/database/migrations/`, `vite.config.js`, and `vercel.json`.

3. Current Development Status

Area	Classification	Evidence and qualification
POS catalog, cart and local checkout	Partially Operational	`js/pos.js`, `js/payment.js`, `js/store.js`; local regression suite exercises independent cart rows and VAT-inclusive math. Records remain browser-local.
Orders	Partially Operational	UI records checkout locally; `api/orders.js` validates but does not save orders; Laravel `OrderController` persists to SQL but is not wired to root deployment.
Payments	Partially Operational	Cash/card/split tender allocation is implemented in `js/payment.js`; no payment gateway/acquirer integration was found. “Card” is a recorded tender label, not a processed card transaction.
Inventory	Partially Operational	Local stock and audit screens exist (`js/kitchen_stock.js`, `js/reports.js`, `js/store.js`); no durable, multi-user stock ledger is connected to production.
Reports and analytics	Partially Operational	Many report renderers exist in `js/reports.js`; seed/demo orders are injected, and Laravel `/api/reports` returns fixed sample summary figures. Financial accuracy against a live ledger is unverified.
Authentication	Partially Operational	Root Node handler uses PIN hashes and role checks, but process-memory sessions are ephemeral; frontend state includes local browser persistence. Laravel Sanctum flow passes local tests but is unconnected to deployed UI.
Authorization	Partially Operational	Role restrictions exist in UI and APIs; the Laravel order and shift tenant scoping defects found here were fixed. Broader authorization and production deployment were not penetration-tested.
Categories and product CRUD	Implemented but Untested	Local CRUD UI and persistence exist in `js/admin.js`; no authoritative server catalog/API was found.
Customers and delivery sources	Partially Operational	Local list and mappings exist; Laravel returns seeded customer types. No customer CRM or delivery aggregator API integration found.
Arabic/English	Partially Operational	Dictionaries and language toggling exist in `js/data.js` and RTL/LTR CSS; mixed or mojibake strings remain in source and README, and full screen-by-screen linguistic review was not performed.
Multi-company/branch	Implemented but Untested	company/branch columns and user fields exist in Laravel schema; a full tenant provisioning/admin lifecycle is absent. Tenant query defects in orders/shifts were fixed and tested.
User/permission administration	Partially Operational	Seeded roles/permissions and API RBAC checks exist; no complete user provisioning interface or production identity lifecycle found.
Returns/refunds	Implemented but Untested	`js/returns.js` exists; no connected payment processor refund API or persistence validation found.
ZATCA/e-invoicing	Implemented but Untested	Receipt code includes TLV QR construction (`js/receipt.js`); no external certification, signing, compliance test evidence, or live ZATCA connection is present. Do not treat the README compliance badge as verified.
External integrations	Not Implemented	No real payment, delivery marketplace, fiscal authority, or hardware bridge credentials/configuration were supplied. Printer support falls back to browser print.

Area	Classification	Evidence and qualification
Backups/recovery	Not Implemented	No deployed database backup schedule, restore runbook, or tested disaster recovery process was found.

4. Verified Functional Features

Orders and cart

The local suite checks independent cart rows per tap, per-line quantities/modifiers, row identity, category navigation preferences, totals, and order creation in the browser state. `js/store.js:351` calculates the taxable portion from a VAT-inclusive gross value. A checkout creates a local order and adjusts local product stock. The root API request is asynchronous, omits the item list and full financial breakdown, and suppresses request errors. Therefore, these tests establish local UI behavior, not durable order processing.

Payments and receipts

Cash, card tender labels, partner-credit policy, split allocation, and change calculations are implemented in `js/payment.js`. No acquirer/PSP SDK or server-side payment capture/refund integration was found. Receipt and print layouts are generated by `js/receipt.js` and `js/printers.js`; physical printer behavior was not tested on hardware. VAT is intended to be inclusive. The `SAMPLE_ORD_1013` fixture in `js/data.js` incorrectly had tax 26.93 and paid/total 206.43 for a gross item sum of 179.50; corrected to tax 23.41 and total/paid 179.50.

Inventory, shifts and reports

Inventory and shift state in the front-end is local. The Laravel API has migrations/controllers for orders, order items, customers and shifts. Its reports endpoint currently returns hard-coded summary values rather than aggregates from saved orders. `js/reports.js` seeds demo orders when storage is empty. Reports should not be used for accounting or operational reconciliation until connected to an authoritative store and verified against source transactions.

Languages and layout

English/Arabic dictionaries and RTL/LTR CSS are present. The automated suite includes layout and selector assertions, but there is no independent accessibility audit, broad device matrix run, or Arabic copy review. Existing user requirements concerning cart/catalog positions, independent item rows, dropdown mappings, VAT-inclusive pricing and navigation order were not intentionally changed.

5. Technology Stack

Layer	Observed technologies
Frontend	HTML, CSS, vanilla JavaScript; Vite 5.4.x build tooling
Root API	Node.js CommonJS handlers under `api/`; custom Vite middleware for local API routing; Vercel static build configuration

Layer	Observed technologies
Separate backend	PHP 8.2; Laravel 12; Laravel Sanctum 4; Eloquent ORM; PHPUnit 11
Database	Laravel migrations support SQLite/MySQL configuration; no production database endpoint or deployed connection was established. Root frontend uses browser `localStorage`.
Browser/PWA	Manifest and service/UI assets exist; offline synchronization to a server database was not found.
External services	No confirmed production payment gateway, marketplace API, email service, remote DB, or fiscal integration.

The backend README describes a Laravel 11 API, while `backend/composer.json` requires Laravel 12. Documentation should be reconciled before handover.

6. Bug Fixes and Testing Results

Verified fixes

1. **Sample VAT/total mismatch:** corrected `SAMPLE_ORD_1013` in `js/data.js` to match VAT-inclusive pricing (gross total 179.50, included VAT 23.41, amount paid 179.50).
2. **Cross-tenant order history:** `backend/app/Http/Controllers/Api/OrderController.php` now scopes the order list to the authenticated user's company and branch.
3. **Cross-tenant shift access:** `backend/app/Http/Controllers/Api/ShiftController.php` now scopes active shift reads and shift close operations to the authenticated user's company and branch.

Regression tests were added to `backend/tests/Feature/PosApiTest.php` for cross-tenant order visibility and shift access/closure.

Checks run

Check	Result
`npm.cmd run build`	Passed with Vite 5.4.21. Warnings remain for classic non-module scripts not being bundled and deprecated CJS Vite Node API usage.
`node verify_pos.js`	574 passed, 0 failed. This is a jsdom/source-level simulation suite, not a production browser or live service test.
`C:\xampp\php\php.exe artisan test --testdox` in `backend/`	12 tests passed, 60 assertions. Tests use in-memory SQLite and seeded fixtures.
`php artisan route:list --path=api`	11 Laravel routes registered locally.
`git diff --check`	Passed.
`npm audit --json`	Could not contact npm registry; vulnerability status unavailable.
`composer audit --no-interaction`	Could not contact Packagist; vulnerability status unavailable.

Check	Result
Live URL check	The provided URL could not be fetched by the available web access tool; no live production behavior is claimed.

No production deployment, push, or database mutation was performed.

7. Security Assessment

Authentication and authorization

The root API's `api/_auth_service.js` stores sessions and rate-limit counters in JavaScript process memory. In serverless/multi-instance operation, requests can reach different processes or cold starts, so a valid login may not be recognized consistently and lockout state is not durable. The Laravel backend uses Sanctum token storage but has not been connected to the supplied deployment. The frontend stores user profile state in `localStorage`; client-side state is not an authorization boundary.

The root Vercel order endpoint validates customer/order-type roles but does not persist orders. The Laravel backend verifies roles on tested paths, but the broader application has not undergone a security penetration test. No security certification is claimed.

Tenant isolation

User, order and shift records have company/branch identifiers. This audit fixed tenant scoping for Laravel order history, active shift reads, and shift closing. Other tenant-aware administrative flows and every data path require a dedicated multi-tenant review before multiple clients are onboarded.

Validation, privacy and integrity

Laravel order writes accept client-provided totals, tax, discount, payment amounts and item prices; server-side canonical catalog price/total reconciliation is not implemented. The root API receives only an order identifier and summary for authorization and silently ignores failures. Browser `localStorage` can be read or modified by the user/device and is not a safe shared ledger. Local sample records contain staff/customer-like names; seed/demo content must be removed or clearly isolated before real operation.

No real customer data, passwords, tokens, or API keys are included in this report. Repository configuration includes seed/test credential material and sample identities; rotate/remove test credentials and use secret-managed production identities before any commercial deployment.

Dependencies, backups and deployment

Automated npm/Composer advisory checks were blocked by unavailable registry network access. The lockfiles pin third-party packages, but current vulnerability status is unknown. No production backup/restore evidence or monitoring/alerting runbook was found. Root Vercel configuration targets a static Vite build; backend database hosting, secrets, migrations, monitoring and retention are unspecified.

8. Database and Architecture

The Laravel schema contains users, cache/jobs, Sanctum tokens, POS shifts, customers, orders and order items. Migrations are under `backend/database/migrations/`. Orders include company and branch keys and JSON raw payload; order items retain item names, prices, quantities, add-ons, modifiers and notes. No catalog/category tables, stock movement ledger, payment transaction ledger, branch registry, or migration history for production data operations was found.

There are two competing architectures:

- **Vercel root:** Vite static frontend plus Node API handlers that use in-memory auth and do not save orders.
- **Laravel backend:** SQL-backed API with migrations and tests, but no root deployment/integration configuration.

This separation is a blocking integration issue for acquisition handover. Select one API/data path, connect the frontend to it, make the database authoritative, and remove or retire duplicate endpoints before production.

9. Third-Party Dependencies and Licences

Root dependencies are declared in `package.json` / `package-lock.json`; notable packages include Vite, jsdom and Puppeteer. Backend dependencies are declared in `backend/composer.json` / `backend/composer.lock`; notable packages include Laravel, Sanctum, PHPUnit and related framework packages. Lockfile entries show common permissive licenses such as MIT and Apache-2.0, but no complete third-party attribution or license inventory was generated as part of this audit. The project root has no `LICENSE` file. Confirm each transitive package license and preserve required notices before redistribution.

Fonts, icon assets, images and copied/generated visual assets should be traced to their source and license. Confirm ownership of all project-specific code and any contractor contributions with the relevant developers and legal counsel. Repository access alone does not prove exclusive ownership or transferability.

10. Hosting and Operating Costs

The current Vercel project/account plan and invoices are not accessible from this repository. For commercial use, Vercel's current documentation describes Pro at **USD 20/month** with usage credit; additional usage may add cost. Hobby is described as non-commercial/personal use only. These are vendor-published plan prices, not a quote for this project. Vercel Pro plan · Vercel fair-use guidelines

No production database provider has been selected. A managed SQL database, backups, monitoring, email/SMS, domain renewal, tax/e-invoicing certification, and payment provider fees would be additional and workload-dependent. A low-traffic planning placeholder of **USD 20–100+/month** for hosting plus a modest managed database is only a rough budget envelope, not a verified price or production recommendation; it excludes payment processing, hardware, tax service, and support labor. Actual cost requires the selected DB, region, retention/backup target, transaction volume, uptime needs, and Vercel usage. The free database tiers are inappropriate to assume as a production service level without reviewing limits and recovery requirements.

Optional hardware may include receipt/kitchen printers and cash drawer integration; current browser print capability has no verified per-device cost. No external API subscriptions are verified in the code.

11. Source Code and Technical Handover

The checkout contains the frontend, root API handlers, Laravel application, SQL migrations, seeders, and local tests. It also includes Vite config, Vercel config, package/composer lockfiles, and basic README documentation. `backend/.env.example` is present; production credentials are not in this report. No deployment guide for the combined system, production topology, database backup/restore instructions, secrets inventory, release procedure, or support runbook was found.

An independent operator would need to obtain a complete repository transfer, verify all branches/history and remote assets, provision hosting and a database, configure secrets, integrate frontend and backend, rotate seeded credentials, set up migrations/backups/monitoring, validate payment and fiscal integrations, and complete acceptance/security testing.

12. Acquisition Readiness

Readiness: Low; not ready for independent production transfer as-is. Source files and schemas are present locally, but the deployed frontend and Laravel backend are not a coherent end-to-end system. Durable production order storage, a reliable session store, tenant onboarding, backup/recovery, verified dependency security, and operational setup remain unresolved. Configuration portability is partial: `.env.example`, lockfiles, and migrations exist, while no complete handover/runbook or production environment declaration exists.

Transferable technical assets appear to include the POS interface, local calculation and catalog flows, printing/receipt layouts, report UI, Laravel scaffold, migrations and automated tests. Transferability is subject to legal confirmation of authorship, contractor assignments, third-party terms, font/image/icon rights, and any repository or hosting account obligations.

No acquisition price, monthly development arrangement, or other commercial transaction is approved or implied by this report.

13. Known Limitations and Risks

- LocalStorage is per browser/device and has no synchronization, central backup, or authoritative cross-device order ledger.
- Vercel root order API only validates and uses process-memory sessions; order persistence and session durability are absent.
- Laravel backend is separate and not integrated with the root frontend or Vercel deployment.
- Laravel accepts client-supplied financial values; server-side pricing/tax reconciliation and payment verification are absent.
- Report routes/data include hard-coded or seeded sample values and are not verified accounting outputs.
- No real payment processor, delivery aggregator, printer bridge, or external fiscal authority integration was validated.
- No live-site inspection was possible in this audit environment.
- Dependency security scans were blocked by registry connectivity.

- No backup/restore test, load test, accessibility review, formal Arabic review, or external penetration test was performed.
- Exclusive ownership and third-party asset rights require legal/developer confirmation.

14. Recommended Next Steps

1. Choose Laravel or another durable backend as the single system of record; integrate all POS checkout, catalog, inventory and report requests with it.
2. Replace process-memory sessions/rate limits with a shared durable store; rotate and remove seeded/test credentials.
3. Validate order payloads server-side and calculate totals from trusted catalog prices, modifiers and VAT rules; persist payment references rather than treating card labels as captured funds.
4. Complete tenant scoping and authorization review across all reads, writes, reports, shifts, users, inventory and administrative endpoints; add cross-tenant regression tests.
5. Remove demo orders and fixed report figures from production; verify all reports against persisted transaction fixtures and reconciliation cases.
6. Establish hosting/database ownership, environment configuration, migrations, backup retention, restore drills, monitoring, incident response and release procedures.
7. Run npm and Composer security audits from a network-enabled environment; produce a complete software bill of materials and license/attribution inventory.
8. Conduct browser/device, payment, printer, accessibility, Arabic-language, load and independent security testing before commercial rollout.
9. Obtain written developer/contractor IP assignments and legal review of third-party assets, data privacy obligations and the proposed source-code transfer scope.

15. Final Technical Conclusion

The codebase demonstrates substantial POS interface work and passes its available local UI and Laravel test suites after the defects described above were corrected. It is best classified as a **prototype / demonstration MVP** with useful reusable frontend and backend components. It is **not production-ready and has low acquisition readiness** until the split architecture is unified, transactions and sessions are made durable, financial/security rules are enforced server-side, demo data is removed, operations are documented, and ownership/licensing are confirmed.

This conclusion is limited to the local repository and checks listed in this report. The live deployment could not be inspected, dependency advisories could not be retrieved, and no production credentials, account billing, database, logs, or customer records were accessed.

Appendix: Reviewed evidence paths

`index.html` ; `js/app.js` ; `js/data.js` ; `js/store.js` ; `js/pos.js` ; `js/payment.js` ; `js/receipt.js` ; `js/reports.js` ; `js/admin.js` ; `api/_auth_service.js` ; `api/orders.js` ; `api/auth/` ; `vite.config.js` ; `vercel.json` ; `package.json` ; `package-lock.json` ; `backend/composer.json` ; `backend/composer.lock` ;

`backend/routes/api.php`; `backend/app/Http/Controllers/Api/`; `backend/app/Models/`; `backend/database/migrations/`; `backend/database/seeders/DatabaseSeeder.php`; `backend/tests/Feature/PosApiTest.php`; `verify_pos.js`.